本形學　Ontypology

從異同到分合，從分合到形，從形到理解這條路，大腦走了億萬年。

邏輯學百年前是哲學，當看清楚後是顯學(科學），今探討「形」(大腦概念）是哲學，本形學將「形」，看清楚轉成顯學(科學），並實踐化 Mind AI 。



本形學研究系列 · IC 設計篇

IC 設計大腦：SiliconMind 與 Verilog 的準確方法

本形學在積體電路設計的精確方法、形式化與時間維度

本場簡報的六個篇章

AGENDA

01

緒論

IC 設計為何需要本形學

02

空間本形基礎

異同→分合→形、容器、SOM、抽象

03

Verilog 形對照

module = 形的精確映射與程式碼範例

04

SiliconMind

雙引擎五層架構與驗證迴圈

05

形式化

公理 5-1 與定理 5-1 ~ 5-3

06

時間本形與案例

十八主題四模組 + FIFO 實證 + 五維評估

01

緒論：IC 設計為何需要本形學

從規格到版圖，每一步都是形的建構

IC 設計是「形」的工程

- 命名訊號與模組——形的標記工作
- 切分模組、歸併功能——分合的動作
- 定義介面——建立形與形的關係
- 模擬比對波形——三層判斷的日常



圖 1-1　本形學的建造鏈：從感知形化到輸出的五層架構

資深與新手的差別：
不在於誰寫得多，而在於誰的「形」立得穩——身份清楚、結構分明、類別規則明確。

現有 EDA 與 LLM 流程的缺口

1.2

EDA 工具鏈

數值完美、形盲

時序算到皮秒、形式驗證窮盡空間；但它不問身份、不問結構、不問類別——形的判斷全落在工程師直覺。

LLM 流程

語感強、兩層缺

擅長抽象層（生成像 FIFO 的程式碼）；缺容器層（命名漂移）與組成層（階層崩壞、埠對不上）。

RAG 檢索

給資料、不給框架

補上文件依據，但沒有三層判斷的座標系——失敗時說不出錯在哪一層。

解方：三層判斷（容器 / SOM / 抽象）+ 雙引擎（LLM + Mind Model）+ 時間迴圈（預測 - 驗證 - 修正）= SiliconMind

SiliconMind：本文四項貢獻

1.3

（一）精確對照

Verilog ↔ 本形學 ↔ RDB 三方對照表，「module 是形」成為逐條可檢查的對應，不是比喻。

（二）形式化

公理 5-1 與定理 5-1 ~ 5-3 及證明：判斷從描述工具升級為可推導的命題系統。

（三）時間方法論

時間本形十八主題聚合為四大模組，對接到規格、RTL、驗證、重構循環。

（四）實證評估

同步 FIFO 全流程案例 + 五維準確性雷達對照，量化雙引擎優勢。

02

空間本形與形邏輯

異同→分合→形：一切推導的認知起點

異同 → 分合 → 形

2.1

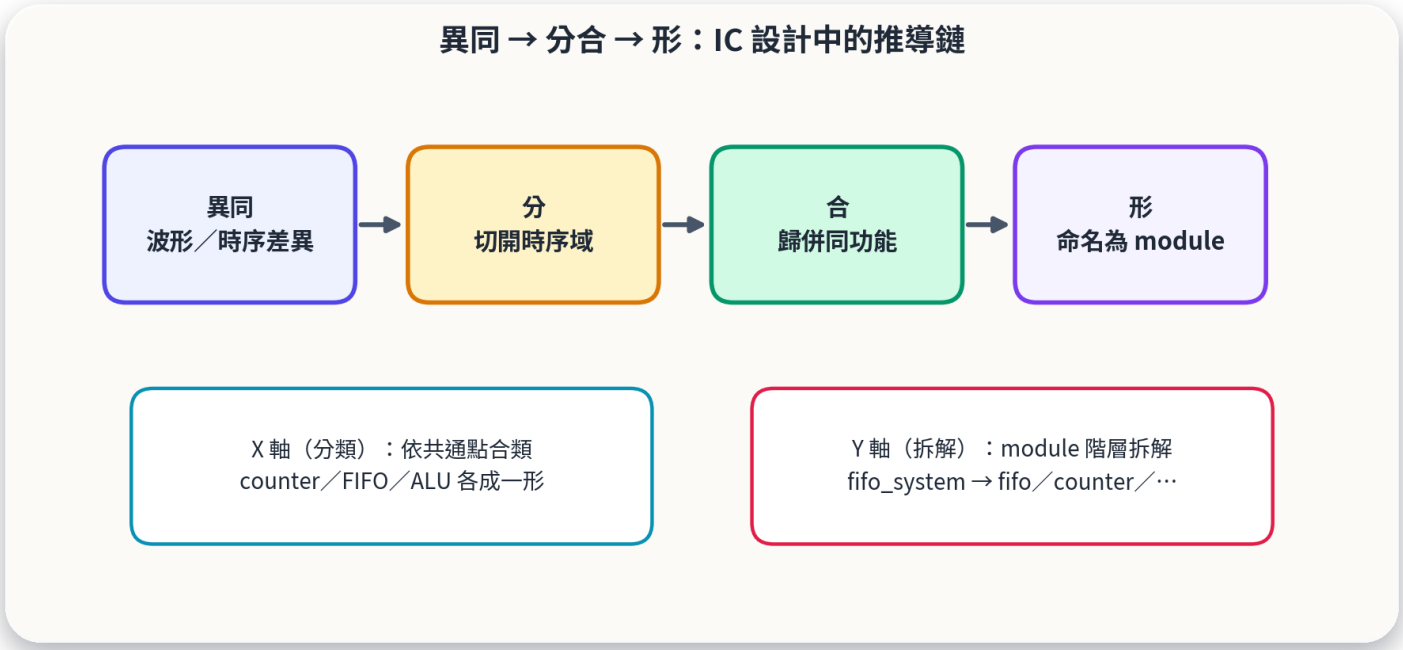


圖 2-1　IC 設計中的推導鏈與 X / Y 兩軸

分

看出這些東西「彼此不一樣」的地方

合

在不一樣裡，再找出「其實可以算同一類」

形

分合畫出的邊界；能認得、能命名的東西

對應格式塔「圖與地」：先有差異，才畫得出邊界

屬性與屬性值：差異逼出維度

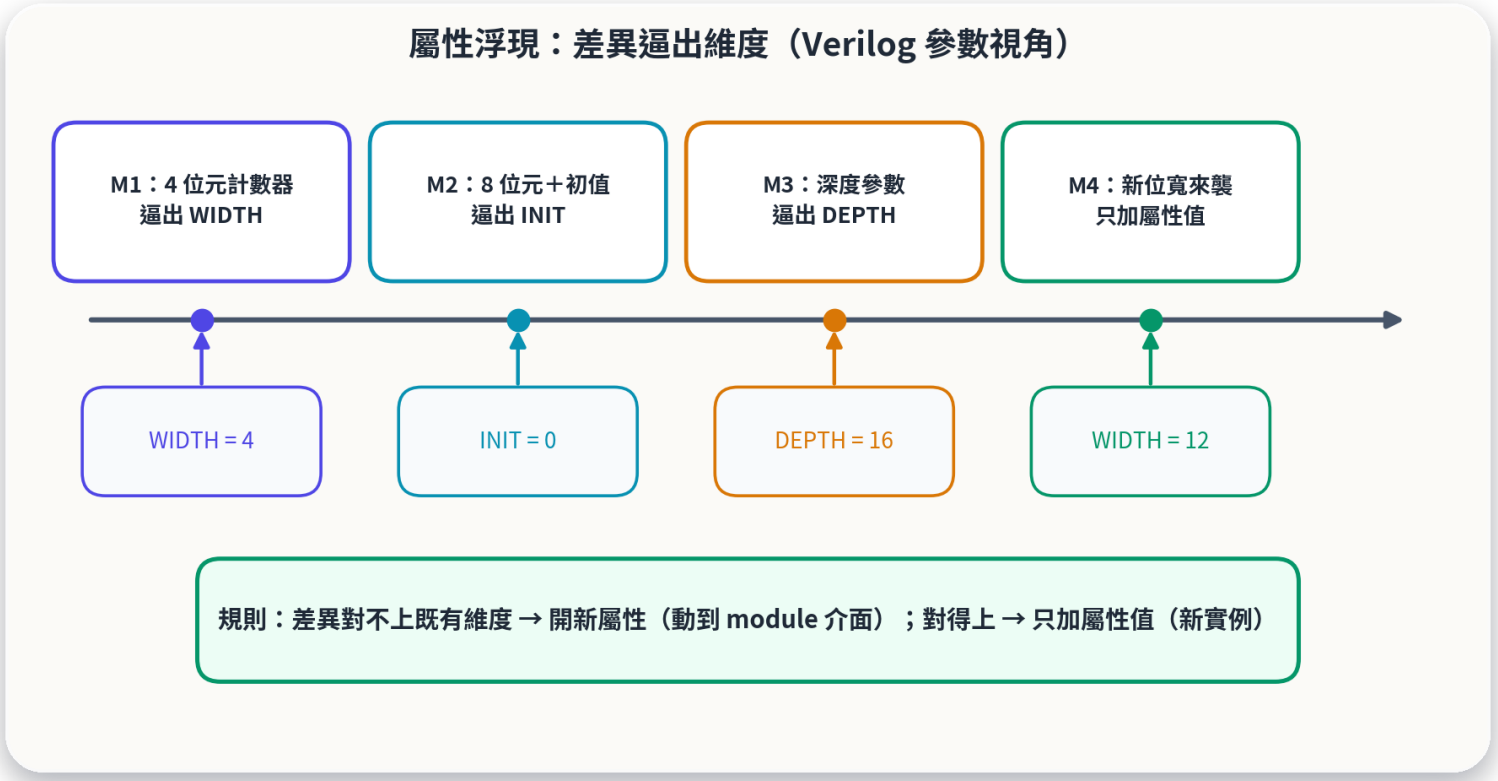


圖 2-2　屬性浮現的時間軸（Verilog 參數視角）

對不上既有維度
→ 開新屬性（改 module 介面）
→ 並回填所有已存在個體

對得上既有維度
→ 只加屬性值（新例化）
→ 不驚動形本身

空間容器與 SOM：身份 × 結構

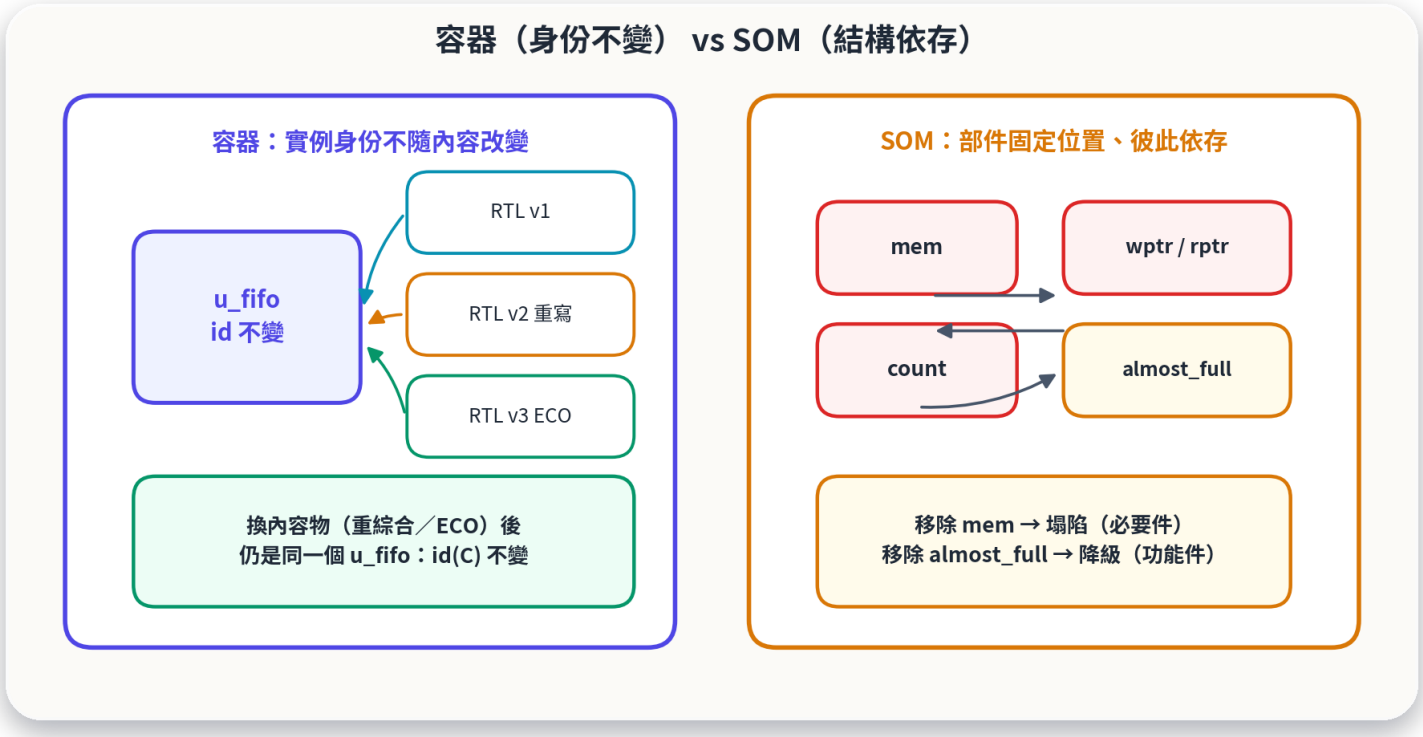


圖 2-3　容器（身份不變）與 SOM（結構依存）

容器 C = (id(C), S(C))

內容可替換，身份不可替換：重綜合、ECO 後，u_fifo 還是 u_fifo。

SOM(M) = (P, R, G)

部件固定位置、彼此依存；remove(p, M) 的後果決定必要 / 降級 / 非必要三級。

形邏輯的三種加值

2.4–2.5

加值類型	括號	補上的性質	IC 設計對應
容器	( )	身份的獨立性	instance 名：重綜合、ECO 後仍是同一個實體
SOM	{ }	元素間的組成結構	module 階層：部件位置固定、彼此依存
抽象	[ ]	生成規則	參數化 IP：通過交集判準 ∩(T) 自動屬於此類

( ) 容器

換了內容物，還是同一個嗎？

{ } SOM

拿掉部件，還是同一個嗎？

[ ] 抽象

新物件，通過規則 ∩(T) 了嗎？

03

Verilog 的形對照：精確方法

module 是形、instance 是實體——可逐條檢查的同構

module 是形、instance 是實體

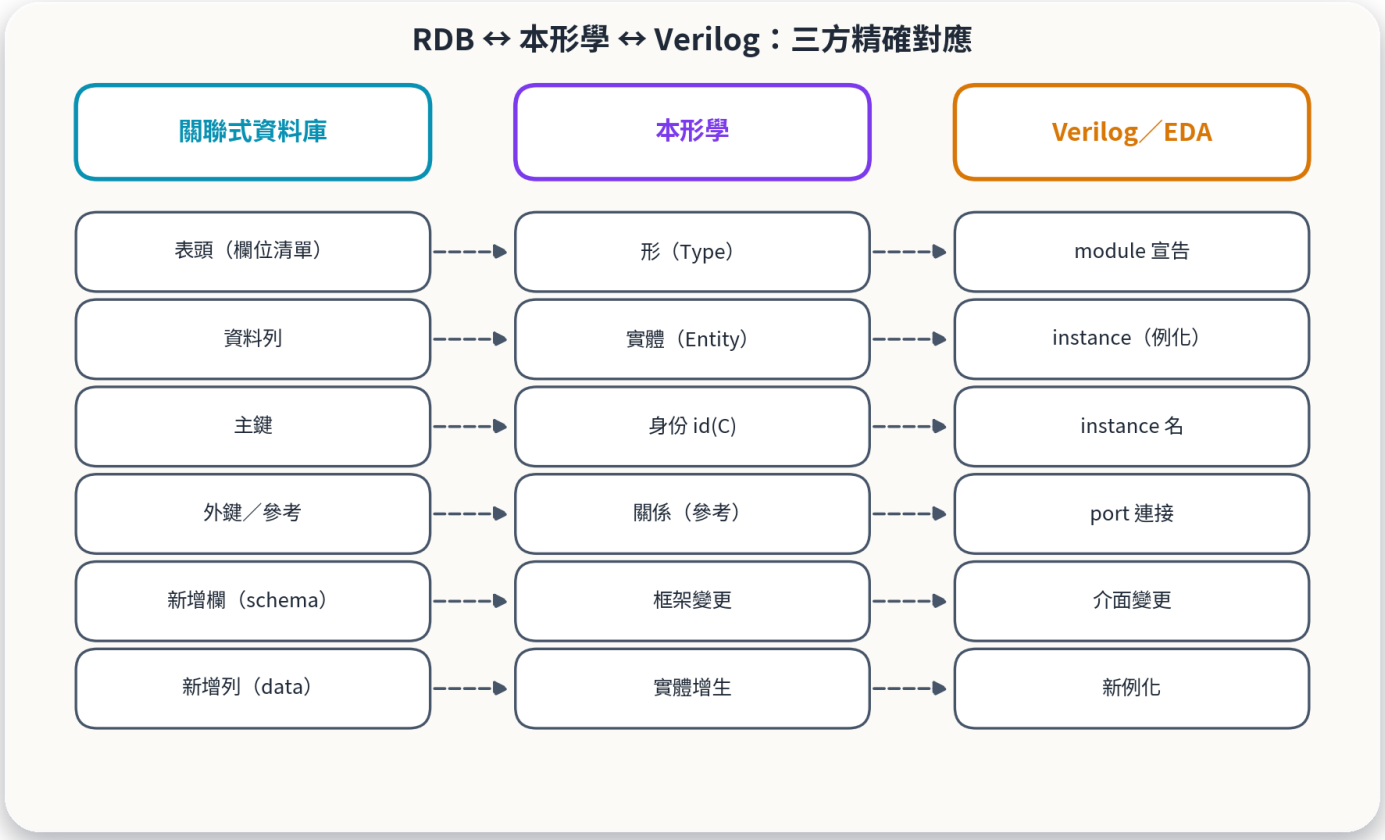


圖 3-1　RDB ↔ 本形學 ↔ Verilog 三方精確對應

module 宣告 = 表頭 = 形

一組問題清單：要接哪些埠、配哪些參數。它不是任何一個電路。

新增例化 vs 改介面：不對稱

新增 instance：形不動、實體多一個（輕）
改 module 介面：形變大、所有例化補值（重）

module 的形解剖：一個宣告，四層形義

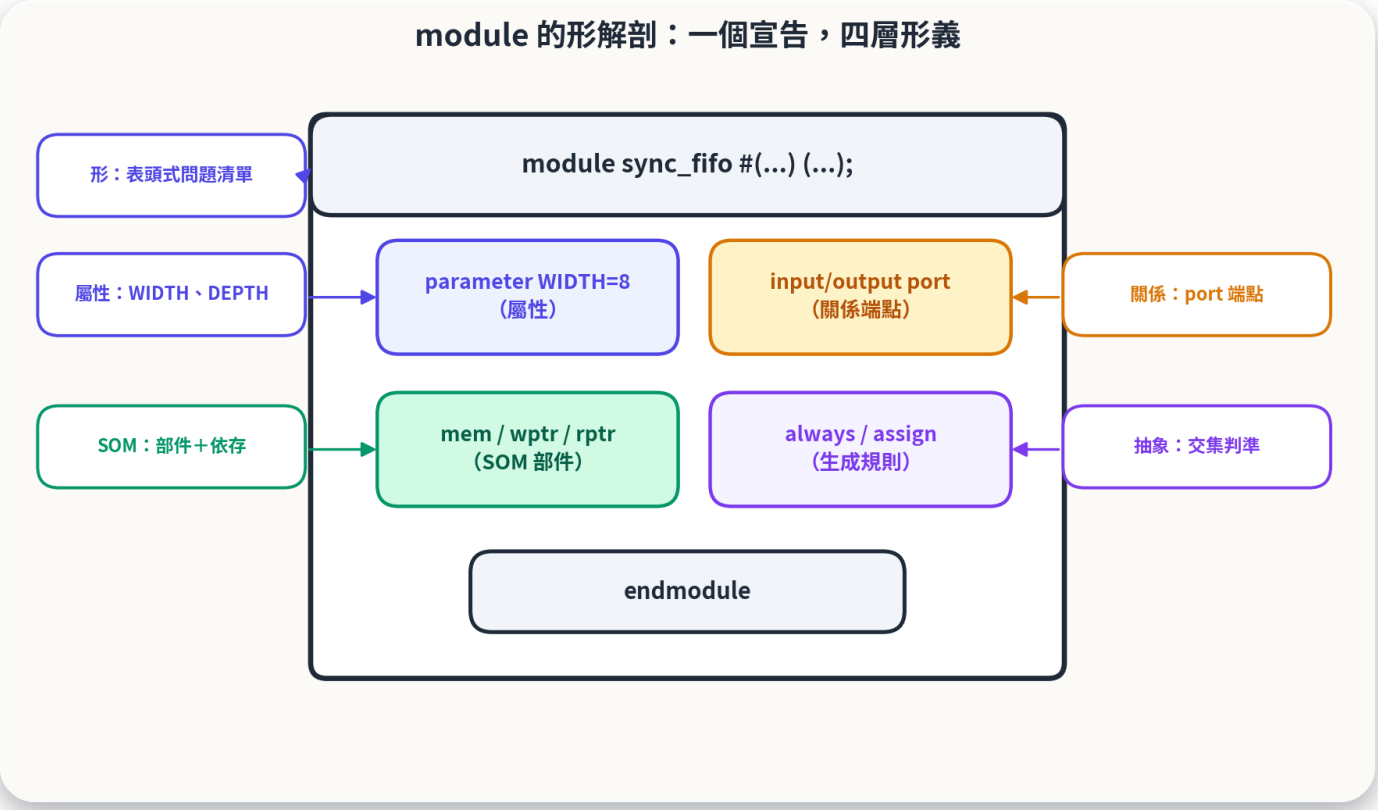


圖 3-2　module 的形解剖

parameter	屬性與屬性值
port 連線	關係的端點化
內部部件	SOM 結構
always / assign	生成規則

精確對照總表（表 3-1 精選）

3.1–3.4

Verilog 構造	本形學概念	RDB 對應	一句話說明
module 宣告	形（Type）	表頭	規定內容長在哪裡的問題清單
parameter	屬性 / 屬性值	欄位 / 欄值	維度固定、刻度可延伸
port 連線	關係	外鍵 / 參考	存指向、不複製內容
instance 例化	實體（Entity）	資料列	具體、可指認、有名字的電路
instance 名稱	身份 id(C)	主鍵	與內容無關、不隨替換改變
module 階層	SOM / Y 軸	主從表	部件拆解與依存分析
generate	框架套用	批次載入	一條規則生成 N 個實體
參數化 IP	抽象類別 [ ]	檢視（view）	開放生成：∩(T) 動態收成員
重綜合 / ECO	內容替換	資料維護	id 不變（容器）

完整範例：框架批量生成實體

程式碼 3-1　參數化計數器（形）

```
module counter #(
  parameter WIDTH = 4,    // 屬性
  parameter INIT  = 0     // 屬性
)(
  input  wire             clk,
  input  wire             rst_n,
  input  wire             en,
  output reg [WIDTH-1:0] q
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n)     q <= INIT[WIDTH-1:0];
    else if (en)    q <= q + 1'b1;
endmodule
```

程式碼 3-2　定時器組（框架＝generate）

```
module timer_bank #(parameter N = 4,
                    parameter WIDTH = 8)
  (input clk, rst_n,
   input  [N-1:0]       en,
   output [N*WIDTH-1:0] q_flat);
  genvar i;
  generate
    for (i = 0; i < N; i = i + 1)
      begin : g_timer
        counter #(.WIDTH(WIDTH))
          u_tmr (                // 實體身份
            .clk(clk), .rst_n(rst_n),
            .en(en[i]),
            .q(q_flat[i*WIDTH
                      +: WIDTH]));
      end
  endgenerate
endmodule
```

04

SiliconMind：雙引擎五層架構

LLM 負責語感與生成，Mind Model 負責形的判斷與驗證

雙引擎五層架構

4.1–4.2

LLM
引擎

Mind Model
引擎

三層
判斷



L1–L2
形化定義

L3–L4
結構驗證

L5
輸出交付

圖 4-1　SiliconMind：IC 設計大腦的雙引擎五層架構

工作占比演化：99% → 80% → 60%

4.2

初期

99%

人做形化，AI 只代筆

工程師時間幾乎全花在 L1～L3 的形化工作

中期

80%

AI 起草、人監督

形定義與結構由 AI 草擬、人審核；驗證迴圈自動化

成熟期

60%

AI 自主形化，人定框架

人退到制定 F(T)、裁決邊界、決定『晶片該是什麼形』

99/80/60 是「人必須親自做形化」的比例——AI 成熟的標誌，是人從代筆中解放、回到框架與價值的決定

時間迴圈：預測 - 驗證 - 修正

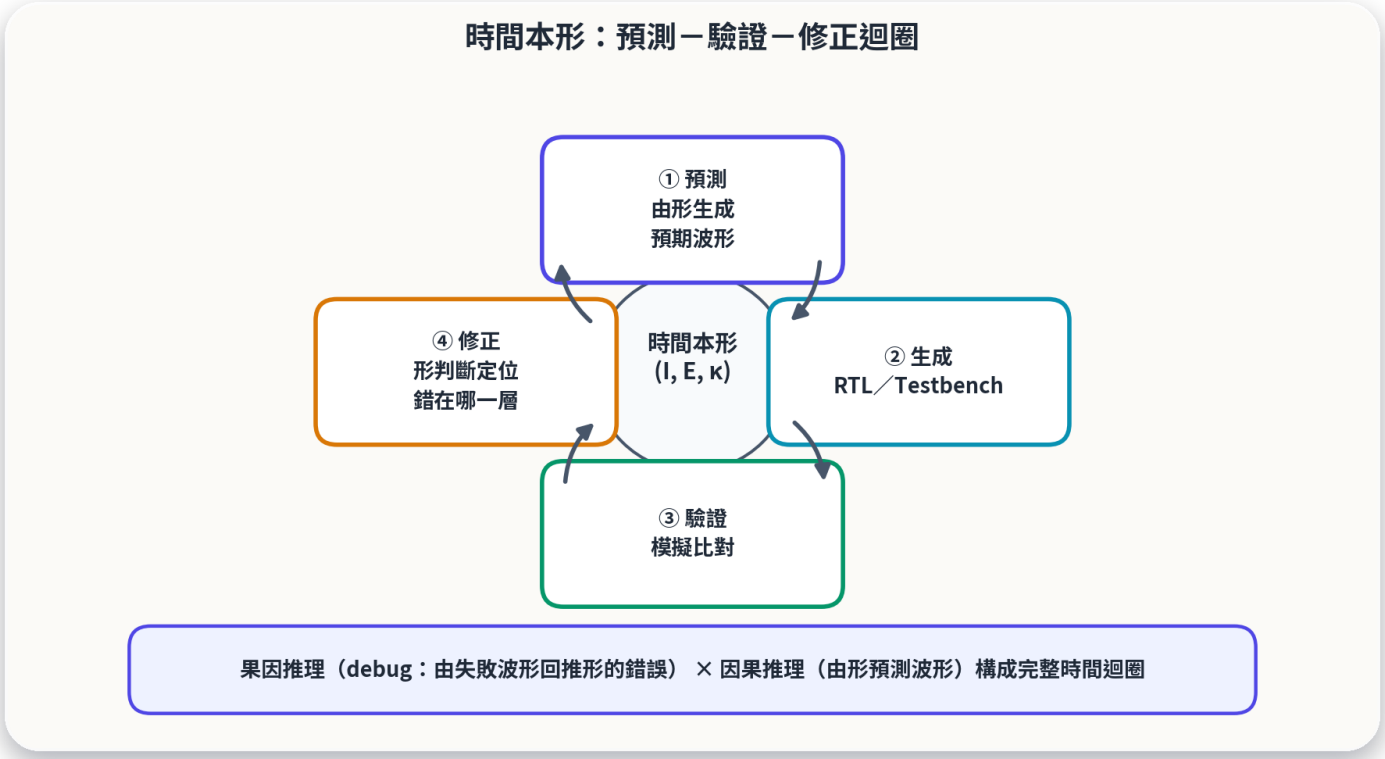


圖 4-2　時間本形：預測 - 驗證 - 修正迴圈

因果推理（前向）

由形預測波形：先寫下「應該長什麼樣」，再生成 RTL 去對。

果因推理（反向）

由失敗波形回推形的錯誤層：容器？SOM？抽象？修正有了座標。

05

形式化：準確方法的定理骨架

一條公理、三條定理，讓「準確」可推導

形式化定理地圖

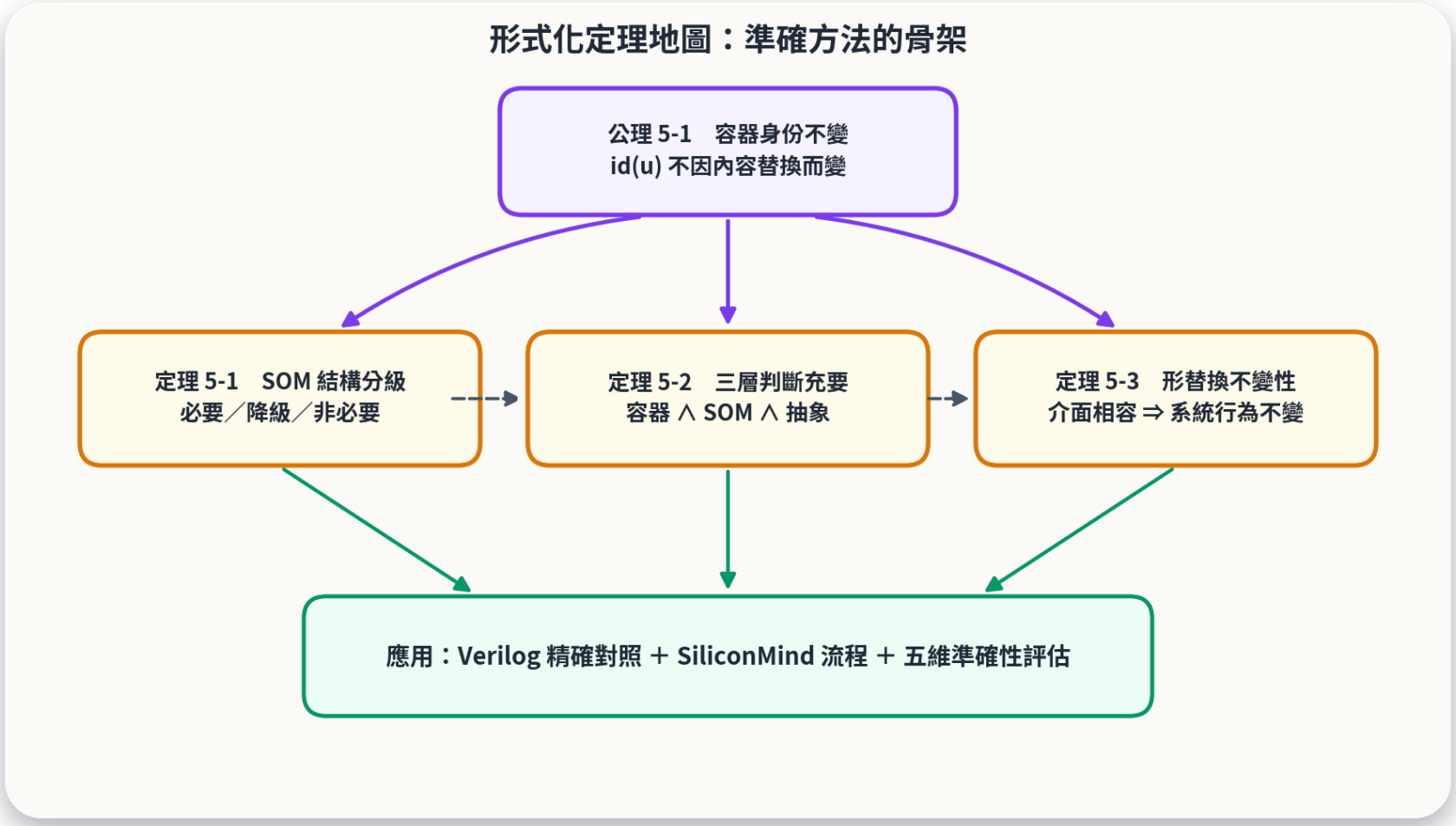


圖 5-1　公理與定理的支撐關係

公理 5-1（容器身份）+ 定理 5-1（結構分級）

5.1–5.2

公理 5-1　容器身份不變

對任一實例 u 與任一內容替換 σ（RTL 重寫、重綜合、ECO）：id(u[σ]) = id(u)。替換變的是內容，不變的是身份。

工程含意：
instance 名是身份憑證。違反它的流程（每次生成重新隨機命名）會讓設計歷史不可追蹤——正是許多 AI 生成流程失敗的深層原因。

定理 5-1　SOM 結構分級

部件 p 對整體同一性的影響恰分三級：

分級	remove(p, M) 的後果	工程對應
必要件	結構塌陷 ≠ M	安全關鍵件
功能降級件	身份存續 ⊂ M	可選功能件
非必要件	完全不影響 ≡ M	裝飾 / 消耗件

三級互斥且窮盡：依 R(M) 中 p 承載的依存角色決定（證明見論文 5.2）。

定理 5-2（三層充要）+ 定理 5-3（替換不變）

5.3–5.4

定理 5-2　三層判斷充要條件

D ∈ [T]　若且唯若

(a) 容器：身份獨立於內容
(b) SOM：必要件齊備、關係完整
(c) 抽象：∀f ∈ F(T)，f(D) = true

單層盲點互補：只看容器，平板也像手機；只看 SOM，計算器也像；只看抽象，對講機也像。

定理 5-3　形替換不變性

A、B 滿足同一 F(T) + 埠關係同構 ψ

⇒ 在任一上層框架 H 以 A 換 B，H 的可觀察行為不變。

證明：三層判斷逐層不變（公理 5-1 + 定理 5-1 + 定義 2-3），由定理 5-2 得形判定不變。

IP 重用的理論基礎：兩顆控制器通過同一組 F(T)，替換不必重驗整個系統——只需驗證 ψ 是否真的保關係

06

時間本形與案例研究

十八主題四模組 × 同步 FIFO 實證 × 五維評估

時間本形十八主題 → 四模組設計循環

6.2

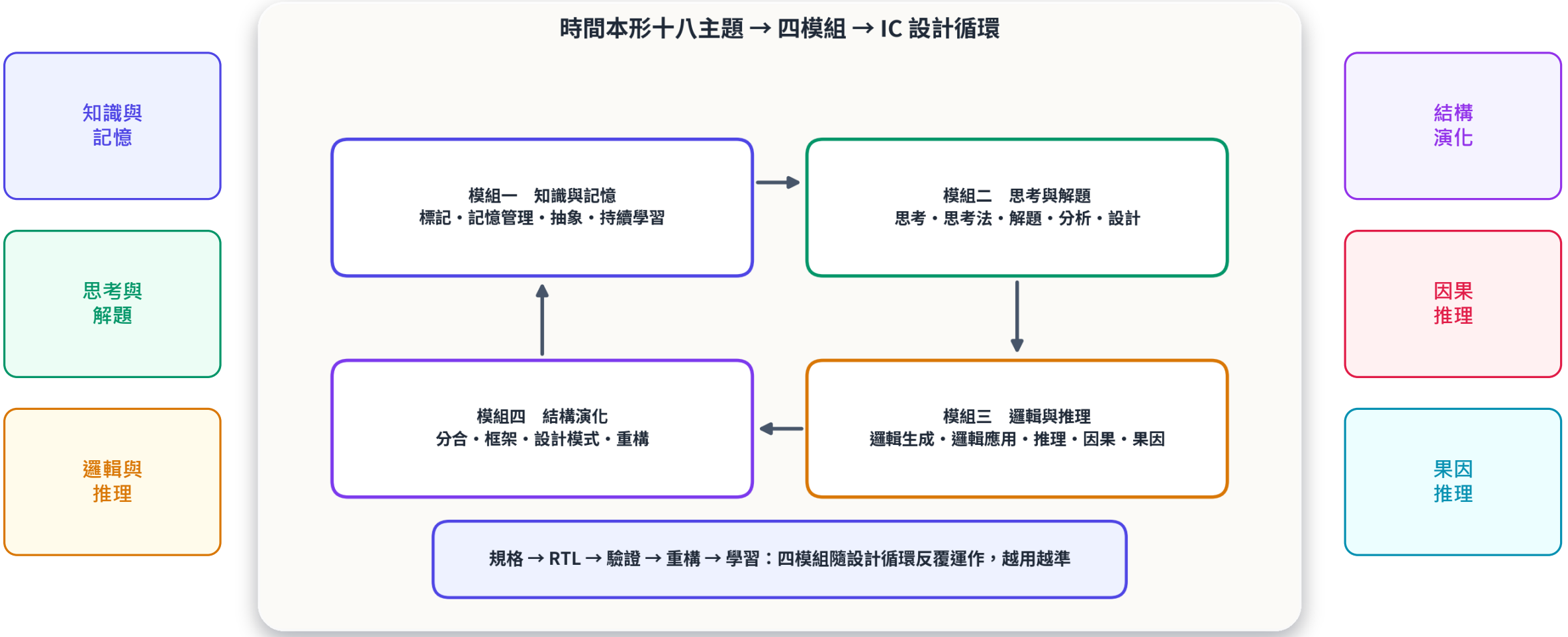


圖 6-1　四模組隨「規格→RTL→驗證→重構→學習」循環運作

案例：同步 FIFO 的五層流程實錄

7.1

層	工作	主導	產出
L1	規格形化：偵測寬度、深度、讀寫協定差異	LLM 起草、Mind Model 確認	FIFO 形候選清單
L2	形定義：WIDTH / DEPTH 屬性、u_dut 身份、port 關係	Mind Model	四元組實例
L3	結構化：mem / wptr / rptr / count 部件分級	Mind Model	SOM 表
L4	形式化與驗證：預測波形、模擬比對、果因修正	雙引擎協同	驗證紀錄
L5	輸出：RTL、測試平台、驗證報告	LLM	可編譯程式碼

類別規則先行：F(FIFO) = { 先進先出序, 可綜合, 介面埠相容, 同步單時脈 }——異步 FIFO、LIFO、環形緩衝被規則排除，邊界由規則決定、不由印象決定。

SOM 部件分級（定理 5-1 判定）

7.3

部件	角色	remove(p, M) 的後果	分級
mem	內容空間	無處存資料，結構塌陷	必要件
wptr / rptr	讀寫位置	無法定位 / 失去先進先出序	必要件
count	容量計數	full / empty 失據，協定崩解	必要件
full / empty	邊界旗標	上下游無法握手	必要件
almost_full	預警旗標	系統照常，僅失去預警	功能降級件
訊號註解	可讀性	完全不影響功能	非必要件

必要件不只意味「不能拿掉」，還意味「它的形必須被完整定義」——初版 count 位寬 [AW-1:0] 的隱性缺陷正是在 L4 迴圈被逼出

L4 實況：三輪時間迴圈

7.4

迴圈	形的預測	模擬結果	果因定位與修正
一	重置後 empty = 1	符合（PASS）	無需修正
二	寫滿 16 筆後 full = 1	不符：count 溢位繞回，full 未拉高	SOM 層：count 位寬不足 → 修正為 [AW:0]
三	讀出序 8'h00、8'h01…	符合（PASS）	無需修正
追加	DEPTH=32 應同樣成立	不符：full 行為錯誤	抽象層：AW 與 DEPTH 約束缺失 → 補約束斷言

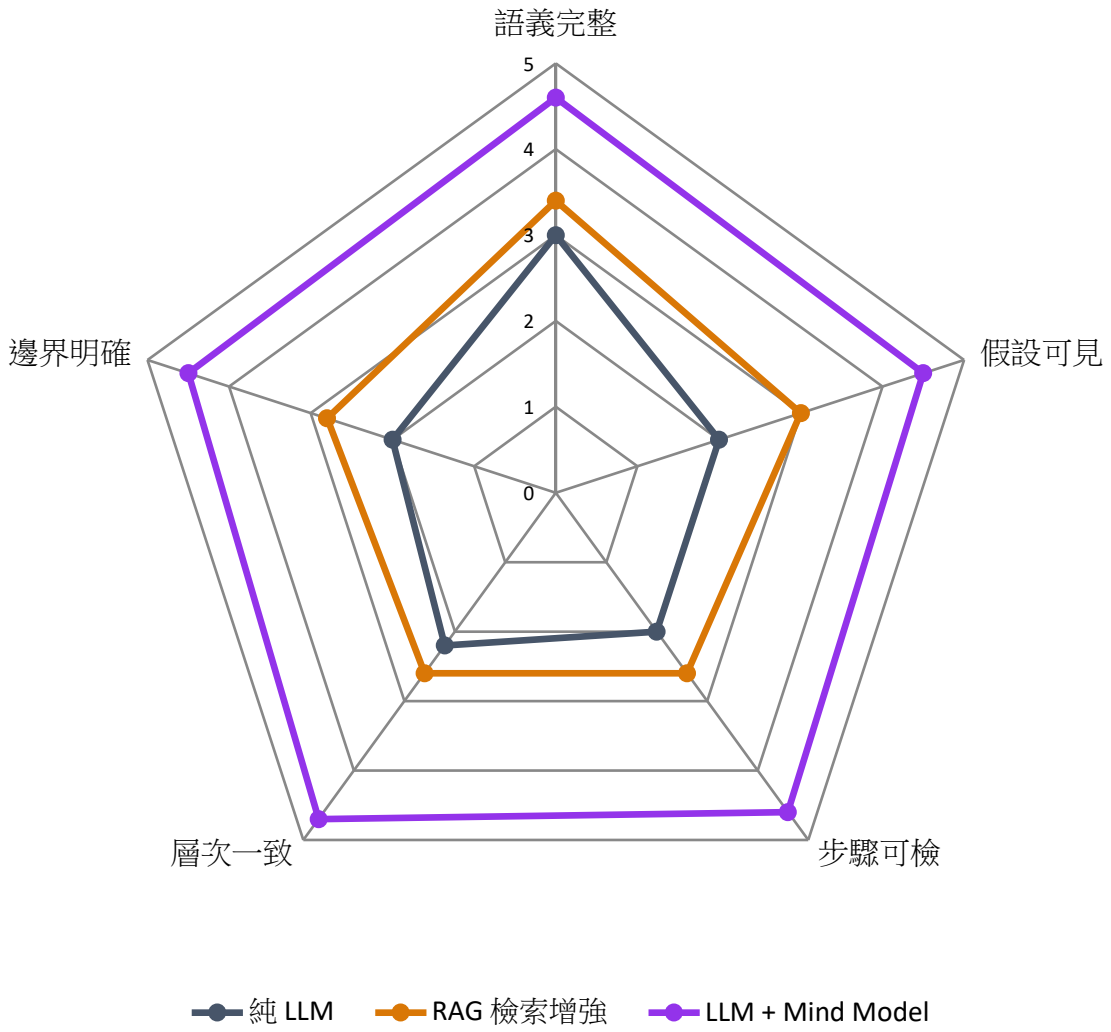
迴圈二教訓

必要件的形要完整：位寬也是形的一部分

追加教訓

屬性間約束本身就是形：F(T) 要寫進去

五維準確性評估：三系統對照



維度	純LLM	RAG	雙引擎
語義完整	3.0	3.4	4.6
假設可見	2.0	3.0	4.5
步驟可檢	2.0	2.6	4.6
層次一致	2.2	2.6	4.7
邊界明確	2.0	2.8	4.5

差距不在生成，在判斷。

語義完整差距最小——雙引擎的領先來自三層判斷的座標系：每一步都可見、可檢、可議。

結論：形，是設計世界的準確方法

第九章

統一

IC 設計、SiliconMind、Verilog 統一到本形學——把工程師本來就在做的命名、分合、分級、判斷，升級為可推導的命題。

落地

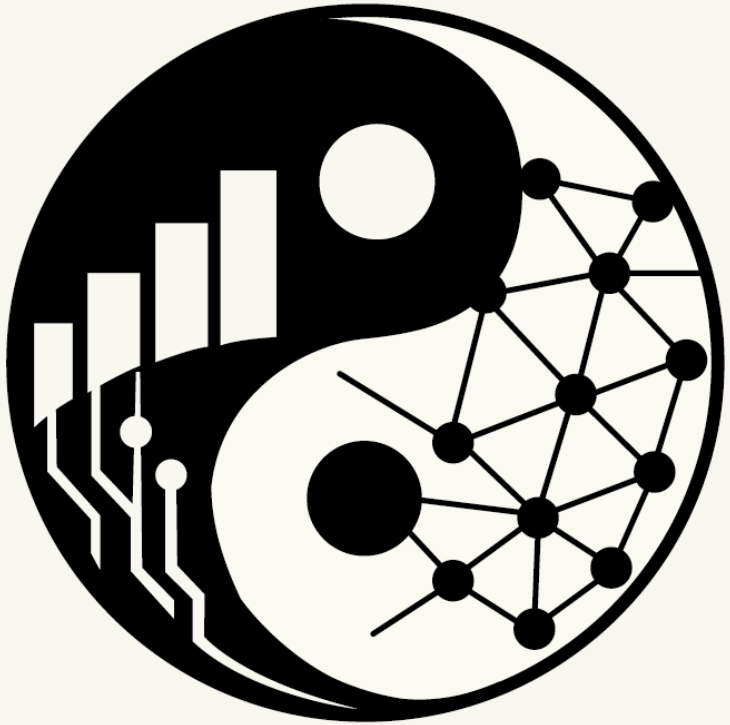
Verilog 逐條對照 + 完整可編譯範例 + FIFO 全流程：形的方法可在 RTL 裡逐行落地。

保證

公理 5-1 與定理 5-1 ~ 5-3：設計對不對、能不能換、該不該收，從直覺變成可檢驗條件句。

未來展望 ① 時間本形完整形式化 ② SystemVerilog / UVM 對照擴充 ③ 三層判斷接入 EDA 流水線 ④ 矽後回饋的持續學習閉環

LonGreat 願景：創造理解世界本質的智慧生命



MIND AI

「讓人工智慧不只是工具，而是真正具備理解、推理、學習與創造能力的智慧生命體系。」

從知識統一，到思維統一，最終邁向智慧統一。這就是 LonGreat 所追求的 AGI 永恆之道。

www.longreat.net

張長豐
gamma@livemail.tw

張梃嶺
mindaiworks@gmail.com